Lecture 15: Memory Hierarchy II — Cache Organization & Direct-Mapped Caches

Session 18 · Wed Oct 28, 2026

Contents

Recap: Lecture 14 . . . 1
Outline . . . 1
An Address Names One Byte . . . 2
Cache Organization . . . 2
Splitting an Address . . . 3
Direct-Mapped Cache (W=1) . . . 3
A Read, Step by Step . . . 3
Tracing a Tiny Cache . . . 4
Why the Index Uses the Middle Bits . . . 4
Cache Misses . . . 5
The Three C's Visualized . . . 5
Cache Parameters . . . 6
Run It: Padding Each Row by One Block . . . 7
AI Systems Connection . . . 7
The Lecture in One Slide . . . 7
Practice . . . 8
Answers to Practice . . . 8
Reading . . . 8

Recap: Lecture 14

A cache is a small, fast memory that keeps copies of blocks from a larger, slower memory. Between main memory and the CPU caches a block is 64 bytes. A read whose block is in the cache is a **hit**. Otherwise it is a **miss**, and the cache fetches the whole block from the level below.

Programs get many hits because they have **locality**: they use the same data again soon (temporal), and they use data next to it soon (spatial).

Summing a 4096×4096 array of `int` column by column was 37 times slower than row by row. The column loop used 4 bytes of each 64-byte block, and the block was evicted before the next column needed it.

Today. How a cache decides where a block goes, how it finds the block again, and why it sometimes evicts a block it still has room for.

Outline

What you will learn today:

1. How the cache finds the block that holds an address
2. How a cache is organized into sets and lines, and what a line holds
3. How the cache splits an address into a tag, a set index and an offset
4. How a direct-mapped cache handles a read, step by step

5. How to trace a sequence of reads through a small cache by hand
6. Why the set index comes from the middle bits of the address
7. The three kinds of misses: compulsory, capacity and conflict
8. The sizes and shapes of the caches in a real laptop
9. A measurement: making each row of a matrix 64 bytes longer makes a column loop 4 times faster
10. How matrix multiplication is split into tiles that fit in the cache

An Address Names One Byte

Memory is a long row of bytes, and each byte has a number, its **address** (Lecture 3). A C pointer holds an address. The instruction `movl (%rdi), %eax` reads the 4 bytes that start at the address in `%rdi`.

Every load and store gives the cache the address it wants. Blocks are 64 bytes long and start at addresses that are multiples of 64, so the cache can work out which block holds the byte, and where in the block it is:

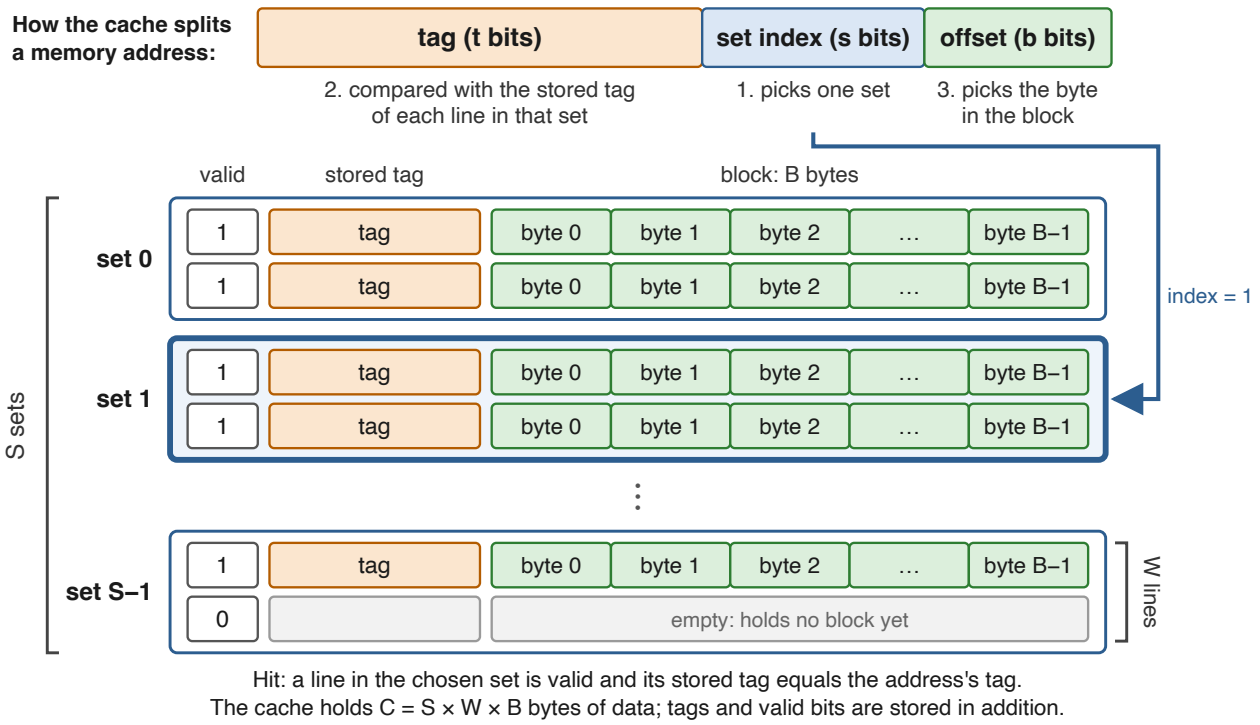
```
block number       = address / 64
position in block  = address % 64

int a[100] starts at address 4096. a[20] is at 4096 + 20 × 4 = 4176:
    block 4176 / 64 = 65, position 4176 % 64 = 16
a[16] to a[31] are all in block 65: after one miss, the rest are hits.
```

The next slides split the block number further, into a set index and a tag.

Cache Organization

A cache has **S sets**. Each set has **W lines**. Each line can hold one block of **B bytes**, plus a **valid bit** that says whether the line holds a block at all, and a **tag** that says which block it is.



Hit: a line in the chosen set is valid and its stored tag equals the address's tag.
The cache holds C = S × W × B bytes of data; tags and valid bits are stored in addition.

Where a block goes: some bits in the middle of its address give a set number, and the block must go into that set. Inside the set, it can go into any of the W lines.

Splitting an Address

The address is an ordinary number; memory knows nothing about these fields. Each cache cuts the address in places set by its own B and S:

Field	Number of bits	What the cache does with it
block offset	b, where B = 2^b	picks the byte inside the block
set index	s, where S = 2^s	picks the set
tag	the rest of the address	compared with the tags of the lines in that set

The offset is the lowest b bits, the index the next s bits, and the tag everything above. In C, for 64-bit addresses:

```
offset = addr & (B - 1);          // the low b bits
index  = (addr >> b) & (S - 1);   // the next s bits
tag    = addr >> (b + s);         // everything above
```

Two addresses in the same block differ only in the offset. Two blocks with the same index go to the same set, and the tag tells them apart.

Direct-Mapped Cache (W=1)

In a **direct-mapped** cache every set has exactly one line, so each address can go into only one place. Here a set is often called a **slot**.

```
Cache: 8 KB, 64-byte lines, direct-mapped
       8192 / 64 = 128 slots, so 7 index bits, 6 offset bits

Address 0x00009064:
  Offset  bits [5:0]    = 0x24 = 36    which byte inside the line
  Index   bits [12:6]   =        65    which of the 128 slots
  Tag     bits [63:13]  =         4    everything above the index

  Look in slot 65. If the stored tag is 4, it is a hit.
  Otherwise it is a miss: fetch the line and store tag 4.

Check, in decimal (0x9064 is 36964):
  36964 % 64          = 36     offset
  (36964 / 64) % 128  = 65     index
  36964 / 8192        = 4      tag
```

A Read, Step by Step

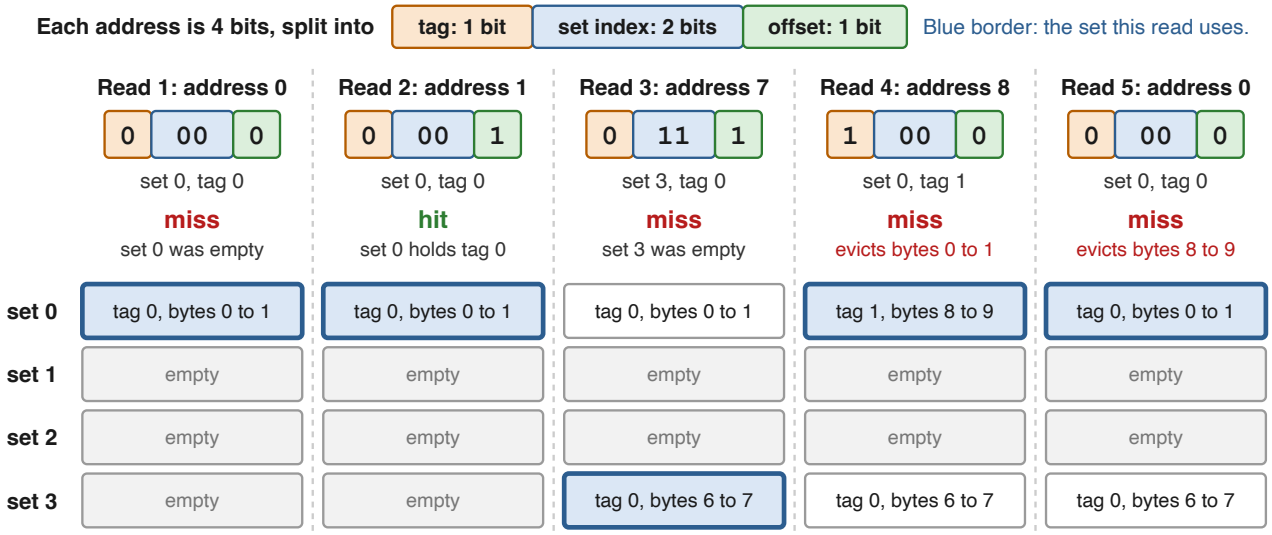
For every read, a direct-mapped cache does the same four steps:

1. **Pick the set.** Take the index bits of the address. They give the slot number.
2. **Check the line.** If its valid bit is 1 and its tag equals the address's tag, the block is in the cache. This is a hit.
3. **On a hit, pick the bytes.** The offset says where in the block the requested bytes start. The cache returns them.
4. **On a miss, replace the line.** The cache fetches the whole block from the level below and stores it in the slot, with the new tag and the valid bit set to 1. Whatever block was in the slot before is evicted. A direct-mapped cache has no choice about which block to evict: the slot has only one line.

The cache never searches other slots. A block that is not in its own slot is not in the cache.

Tracing a Tiny Cache

Take a cache small enough to trace by hand: 4-bit addresses, 4 sets, one line per set, 2-byte blocks. Then b = 1 offset bit, s = 2 index bits, and t = 1 tag bit. The program reads addresses 0, 1, 7, 8, 0:



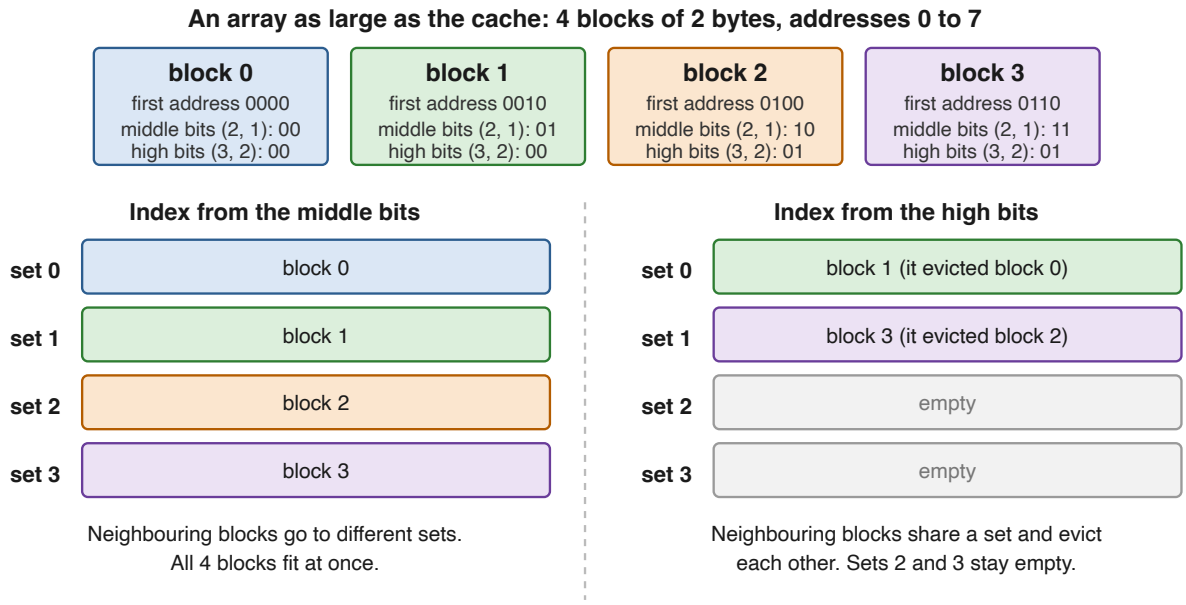
Sets 1 and 2 stay empty. The last read misses although the cache has room for two more blocks: addresses 0 and 8 have the same set index, 00, so they take turns in set 0.

Read	Bits (tag, index, offset)	Set	Tag	Result	Set 0 holds	Set 3 holds
0	0 00 0	0	0	miss	bytes 0 to 1	empty
1	0 00 1	0	0	hit	bytes 0 to 1	empty
7	0 11 1	3	0	miss	bytes 0 to 1	bytes 6 to 7
8	1 00 0	0	1	miss, evicts bytes 0 to 1	bytes 8 to 9	bytes 6 to 7
0	0 00 0	0	0	miss, evicts bytes 8 to 9	bytes 0 to 1	bytes 6 to 7

Sets 1 and 2 stay empty the whole time. The last read misses although the cache has room for two more blocks: addresses 0 and 8 have the same index, so they take turns in set 0.

Why the Index Uses the Middle Bits

As a loop moves along an array, the middle bits of the address change and the high bits mostly do not. So every real cache takes its index from the middle bits. Here is the tiny cache's array of 4 blocks, placed both ways:



A loop that walks an array reads blocks that are next to each other. With the index in the middle bits, neighbouring blocks go to different sets, so an array as large as the cache fills every set. With the index in the high bits, neighbouring blocks go to the same set and evict each other while the other sets sit empty. For all 8 blocks of the tiny cache's 16 bytes of memory: with the middle bits, blocks 0 to 7 go to sets 0, 1, 2, 3, 0, 1, 2, 3; with the high bits, to sets 0, 0, 1, 1, 2, 2, 3, 3.

Cache Misses

Three types of misses:

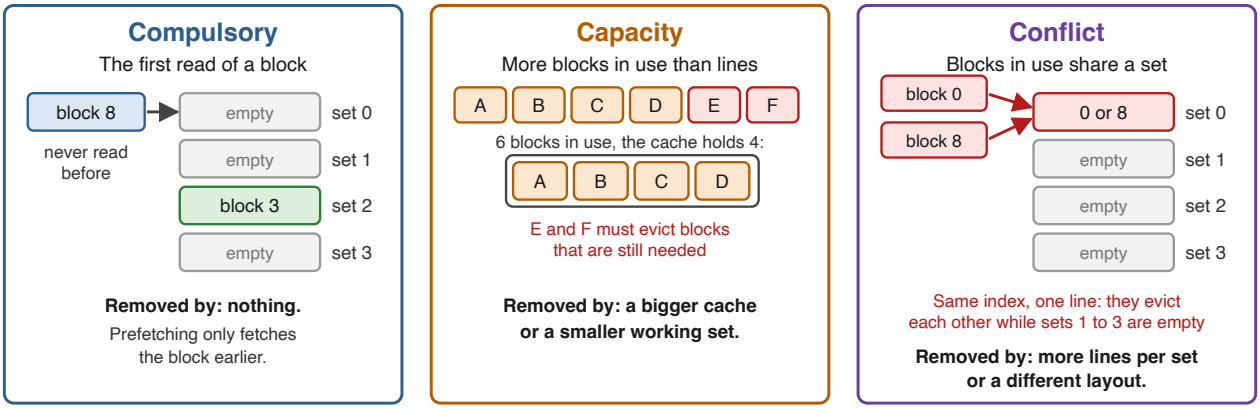
- **Compulsory (cold) miss.** The first access to a block. Nothing can avoid it.
- **Capacity miss.** The working set is larger than the cache.
- **Conflict miss.** Several addresses map to the same set, so they evict each other even though the cache has room elsewhere. Happens in direct-mapped and low-associativity caches.

```c
int a[1024];   // 4 KB direct-mapped cache, 64 slots of 64 bytes.
int b[1024];   // b starts right after a, 4096 bytes later.
for (int i = 0; i < 1024; i++)
    sum += a[i] + b[i];   // a[i] and b[i] map to the same slot.
                          // Each iteration evicts the other's line.
```

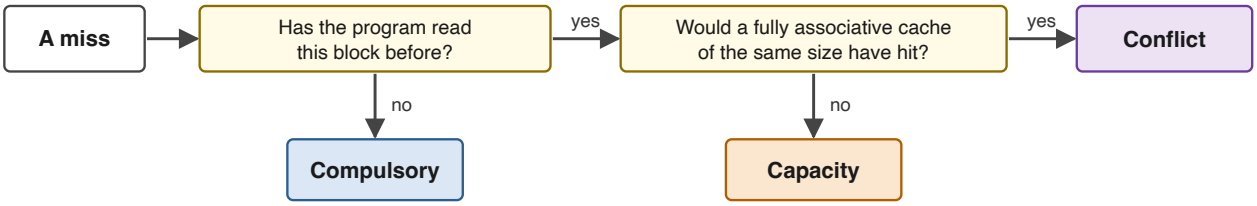
All 2048 reads miss. This is called **thrashing**. With 16 unused `int`s (one block) between the arrays, each block of `b` moves to the next slot, and only the first read of each block misses.

The Three C's Visualized

Each kind of miss has a different cause and a different fix. A **fully associative** cache is one in which any block can go in any line.



To decide which kind a miss is:



Miss type	Cause	What removes it
Compulsory	the first read of a block	nothing; prefetching only hides it
Capacity	working set too large	a bigger cache or a smaller working set
Conflict	blocks in use share a set	more lines per set or a new layout

To decide which kind a miss is, ask two questions in order. Has the program ever read this block before? If not, it is compulsory. Would the read hit in a fully associative cache of the same total size? If it would miss there too, the miss is capacity. If it would hit, the miss is conflict.

In the tiny cache, the first read of 8 is compulsory. The second read of 0 is a conflict miss: a 4-block cache that could put 0 and 8 in different lines would have hit.

Cache Parameters

The laptop from Lecture 14 (AMD Ryzen 7 8845HS) reports its caches like this:

```
$ lscpu -C
NAME ONE-SIZE ALL-SIZE WAYS TYPE        LEVEL  SETS PHY-LINE COHERENCY-SIZE
L1d       32K     256K    8 Data            1    64        1             64
L1i       32K     256K    8 Instruction     1    64        1             64
L2         1M       8M    8 Unified         2  2048        1             64
L3        16M      16M   16 Unified         3 16384        1             64
```

WAYS is W, the lines per set, and **COHERENCY-SIZE** is the block size. For the L1 data cache, S = 32768 / (8 × 64) = 64 sets:

Cache	Size	Lines per set (W)	Sets (S)	Offset bits	Index bits
L1 data	32 KB	8	64	0 to 5	6 to 11
L2	1 MB	8	2048	0 to 5	6 to 16

Cache	Size	Lines per set (W)	Sets (S)	Offset bits	Index bits
L3	16 MB	16	16384	0 to 5	6 to 19

None of these caches is direct-mapped. Each set of L1 can hold 8 blocks with the same index at once. Reading L1 takes about 4 cycles (0.8 ns).

Run It: Padding Each Row by One Block

`01-conflict.c` runs Lecture 14's column loop on two arrays. The rows of `p` end with 16 unused `int`s, so they are 64 bytes longer:

```
static int a[4096][4096];         // rows 16 KB apart
static int p[4096][4096 + 16];    // rows 16 KB + 64 bytes apart

for (int j = 0; j < 4096; j++)    // the same loop sums a, then p
    for (int i = 0; i < 4096; i++)
        sum += a[i][j];
```

```
$ ./01-conflict          # Linux, AMD Ryzen 7 8845HS
run 1: rows 16 KB apart  113.4 ms, rows 16 KB + 64 B apart   26.6 ms,  4.3x
run 2: rows 16 KB apart  111.1 ms, rows 16 KB + 64 B apart   26.6 ms,  4.2x
run 3: rows 16 KB apart  110.9 ms, rows 16 KB + 64 B apart   26.5 ms,  4.2x
```

In `a`, a column's 4096 blocks have the same bits 6 to 11, so they share one L1 set and evict each other. In `p`, each row's block lands one set further on, and the column spreads over all the sets.

Both loops do the same additions on the same values. With rows 16 KB apart, `a[0][j]`, `a[1][j]`, `a[2][j]`, … have the same bits 6 to 11, so all 4096 blocks of a column go to one L1 set of 8 lines. They also crowd into a few sets of L2 and L3. When the loop moves to column `j + 1`, which uses the same blocks, almost all of them have been evicted. With 64 more bytes per row, each row's block lands one set further on, and a column's 4096 blocks (256 KB) spread over all the sets of L2. The next column finds them there.

AI Systems Connection

Almost all the arithmetic in training and running a neural network is matrix multiplication. C = A × B for n × n matrices does n^3 multiply-adds. The plain three-loop version reads B down a column in its inner loop, so for large n almost every read of B misses.

Blocking, also called **tiling**, splits each matrix into small square tiles. The program multiplies one tile of A by one tile of B, adds the result into one tile of C, and only then moves on. The tile size is chosen from the cache size, so that the three tiles fit at once. Three 32 × 32 tiles of 4-byte `float` are 3 × 4 KB = 12 KB, which fits in a 32 KB L1. Each element of a tile is loaded into the cache once and then used 32 times before it is evicted.

GPU matrix-multiply code does the same thing. Each group of GPU threads copies one tile of A and one tile of B into a small, fast memory on the GPU chip, multiplies them, and then loads the next pair of tiles. Libraries such as cuBLAS choose their tile sizes from the sizes of these memories. The same plan works for any loop nest that reads one matrix by rows and another by columns, such as a transpose, `B[j][i] = A[i][j]`.

The Lecture in One Slide

- A cache has S sets of W lines. Each line holds a valid bit, a tag and one block of B bytes.
- An address splits into a tag, a set index and an offset. The index picks the set, the tag says which block is in a line, and the offset picks the byte.
- In a direct-mapped cache (W = 1) each block has exactly one slot, and a miss always evicts the block in that slot.
- Misses are compulsory (first use), capacity (the working set is too big) or conflict (blocks in use compete for the same set).

- Data whose addresses differ by a large power of two lands in the same set. Padding by one block spreads it over all the sets.

Practice

1. A direct-mapped cache holds 16 KB in 32-byte blocks, and addresses are 32 bits. How many sets does it have? How many offset, index and tag bits?
2. For the 8 KB direct-mapped cache with 64-byte lines on the "Direct-Mapped Cache (W=1)" slide, find the offset, index and tag of address `0x12345`.
3. In the tiny cache from "Tracing a Tiny Cache", the cache starts empty and the program reads addresses 3, 4, 11, 3, 12, 5, 2. For each read, give the set, the tag, and whether it hits or misses. Label each miss compulsory or conflict.
4. `float x[8]` starts at address 0 and `float y[8]` starts at address 32. The cache is direct-mapped with 2 sets and 16-byte blocks. A loop computes `sum += x[i] * y[i]` for i = 0 to 7. Which set does each of `x[0]`, `y[0]`, `x[4]` and `y[4]` go to? How many of the 16 reads miss? How many miss if `x` is declared `float x[12]`, so that `y` starts at address 48?
5. Name the kind of each miss: (a) the first read of an array that was just allocated; (b) a loop sums a 64 KB array over and over, and the cache is 32 KB and fully associative; (c) in the "Cache Misses" loop, the read of `a[1]`.
6. On the laptop's L1 (32 KB, 8 lines per set, 64-byte blocks), are two addresses 4096 bytes apart in the same set? 4160 bytes apart? How many blocks, all 4096 bytes apart, can L1 hold at once?

Answers to Practice

1. 16384 / 32 = 512 sets. Offset 5 bits, index 9 bits, tag 32 − 14 = 18 bits.
2. `0x12345` is 74565. Offset 74565 % 64 = 5. Index (74565 / 64) % 128 = 1165 % 128 = 13. Tag 74565 / 8192 = 9.
3. 3: set 1, tag 0, compulsory miss. 4: set 2, tag 0, compulsory miss. 11: set 1, tag 1, compulsory miss, evicts bytes 2 to 3. 3: set 1, tag 0, conflict miss. 12: set 2, tag 1, compulsory miss, evicts bytes 4 to 5. 5: set 2, tag 0, conflict miss. 2: set 1, tag 0, hit. Six misses, one hit.
4. Block number = address / 16, set = block % 2. `x[0]` (address 0) and `y[0]` (address 32) both go to set 0; `x[4]` (16) and `y[4]` (48) both go to set 1. Each read of `y[i]` evicts the block `x[i+1]` needs, and the other way round, so all 16 reads miss. With `float x[12]`, `y[0]` is at 48 (set 1) and `y[4]` at 64 (set 0), so `x` and `y` never share a set at the same time. Only the first read of each of the 4 blocks misses: 4 misses, 12 hits.
5. Part (a) is compulsory. Part (b) is capacity: even a fully associative cache cannot hold 64 KB in 32 KB. Part (c) is conflict: the block of `a[1]` was loaded by `a[0]`, then evicted by `b[0]`, while other slots were free.
6. Yes: 4096 = 2^12, so the two addresses have the same bits 6 to 11. No: 4160 = 4096 + 64, so the index differs by 1. L1 can hold 8 such blocks, one in each line of that set.

Reading

- CS:APP §6.4.1–6.4.2 (Generic cache memory organization, direct-mapped caches)
- The sample code in `sample code/lecture15/`